

## Response to Reviewers — SENSL-26-06-RL-0906

### “Wire-Level Interrupt-to-Decision Latency of On-Sensor MLC versus Host Inference on the NVIDIA Jetson Orin Nano: A Pre-Registered Measurement Study”

We thank both reviewers for a thorough and technically demanding review. Every substantive point raised has been addressed, either by a correction or expansion in the manuscript, or — where a point concerns material outside the paper’s 4-page limit — directly in this response. We address each point in order below, referencing the specific section of the revised manuscript where applicable.

---

#### Response to Reviewer 1

##### Point 1 — Data-rate / window-length confound

*“The host pipeline consumes accelerometer data at 208 Hz... whereas the MLC operates at 26 Hz... This is a first-order confound, and it cannot be resolved by rephrasing the text. It requires a re-run experiment with the host decimated to a 26 Hz equivalent.”*

**Response:** The reviewer’s reading exposed a documentation error in our manuscript, not a rate mismatch in the deployed experiment. No re-run was performed, because the existing campaign already is the rate-matched comparison requested.

This is verifiable in the public record, not a post-hoc assertion. The 208→104 Hz decimation policy was present in the parity tooling on 2026-05-21 (a5a0c892, inline in `replay_parity.c: sensor_odr_hz = 208, mlc_odr_hz = 104, decimation_ratio = 2`), and the live host binary was refactored onto the shared `parity_core.c / pc_step()` core ten minutes later (467e1f5, “refactor: extract parity\_core, slim both binaries”). The deployed values were fixed before data collection: silicon register writes 0x60=0x35 and 0x10=0x50 in `code/mlc_config/mlc_motion_w75.h` by cffe3b4b (2026-05-22), and host parity values `window_length=75, sensor_odr_hz=208, mlc_odr_hz=104, decimation_ratio=2` in `code/mlc_config/tree_w75.json` by c15b2c71 (2026-05-24) — all before confirmatory data collection began (2026-05-26) and before the manuscript’s printed submission date (May 28, 2026). The MEMS Studio export (`mlc_settings.json`, 2026-05-22) records `accelerometer_odr: 208 Hz, gyroscope_odr: 26 Hz, and mlc_odr: 104 Hz` side by side. Independently, the wire-level INT1 pulse width on the MLC pipeline is 9,007.9  $\mu$ s median (IQR 9,006.4–9,009.1  $\mu$ s;  $n = 1,669$  pulses across the 27 confirmatory mlc blocks), consistent with the 104 Hz configuration and far from the 38.5 ms period of a 26 Hz configuration. AN5259 caps MLC ODR at 104 Hz absolutely and requires sensor ODR  $\geq$  MLC ODR; with `CTRL1_XL=0x50` (208 Hz), the 104 Hz MLC configuration is valid and maximal.

Init-phase I<sup>2</sup>C decode of the archived raw captures (evidence package, Zenodo DOI 10.5281/zenodo.22950121; stdlib-only decoder applied to the first ~8 s of each capture) shows the register writes that configured the sensor in both pipelines: `MD1_CFG (5Eh)=0x02, EMB_FUNC_ODR_CFG_C (60h)=0x35`, and `CTRL1_XL (10h)=0x50`. Per the LSM6DSOX datasheet, `MLC_ODR[1:0]` occupies bits [5:4] of `EMB_FUNC_ODR_CFG_C`; `0x35 = 0b00110101` gives `MLC_ODR = 0b11 = 104 Hz` (AN5259 Table 1). `CTRL1_XL=0x50` sets the accelerometer to **208 Hz**. The confirmatory campaign used the same pinned configuration header (`mlc_motion_w75.h`) flashed before every block; we do not claim a separate SDA/SCL decode for the May confirmatory blocks.

The host parity path implements the matching decimation in software: `host_pipeline_parity.c` streams the accelerometer at 208 Hz, while `parity_core.c:pc_step()` returns a decision only

every `decimation_ratio=2` sensor samples and only at each `window_length=75` post-decimation boundary (`tree_w75.json: sensor_odr_hz=208, mlc_odr_hz=104, decimation_ratio=2, window_length=75`). Both pipelines therefore decide on 75-sample windows at 104 Hz — nominal period  $75/104 \text{ s} = 721.2 \text{ ms}$ . The measured  $\sim 706 \text{ ms}$  MLC cadence is the full 75-sample window at the actual ODR (effective  $\sim 106 \text{ Hz}$ , within ODR tolerance), not “one quarter” of a 2.88 s window; that quarter-window explanation is retracted.

The erroneous 26 Hz / 2.88 s text came from ST’s activity-recognition example documentation (“The MLC runs at 26 Hz, computing features on windows of 75 samples”) and the MEMS Studio gyroscope\_odr: 26 Hz field, neither of which is the MLC\_ODR field of `EMB_FUNC_ODR_CFG_C`. The deployed configuration had been deliberately modified from ST’s original `.ucf` (`60h: 0x15 → 0x35, 10h: 0x28 → 0x50`) under the 2026-05-22 pre-registration amendment, but the manuscript and parts of the pre-registration text were never updated to match. We own that inconsistency and have corrected it in the revision. This correction and the withdrawal of the planned 26 Hz host-decimation re-run are recorded pre-collection in amendment v7.15 (Zenodo DOI 10.5281/zenodo.22952272); no data were collected under the withdrawn v7.14 plan (Zenodo DOI 10.5281/zenodo.22951758).

**Manuscript changes:** §III.A (MLC ODR corrected to 104 Hz with register-level provenance), §III.B (host 2:1 decimation and matched 75-sample/721.2 ms windows stated explicitly), §V.C and Introduction contribution 3 (706.5 ms cadence explained as the full 104 Hz window; quarter-window derivation removed), graphical abstract checked for rate claims.

---

## Point 2 — mlc-binary idle multimodality

*“The control (MLC-binary) pipeline shows unexplained, load-dependent multimodality (slower at idle than under contention)... This anomaly... demands systematic investigation.”*

**Response:** We conducted a targeted follow-up investigation testing process-affinity pinning and IRQ-routing as candidate causes. Per our pre-registration’s standing rule (v7.13), analysis branches are timestamped in our Zenodo chain either before the data they interpret or explicitly labeled as post-hoc; here, the investigation’s data were collected 2026-09-21, and the corresponding amendments (v7.12, DOI 10.5281/zenodo.22907481; correction v7.13, DOI 10.5281/zenodo.22907600) were formalized into the chain immediately afterward, on 2026-09-22. We label this analysis branch as post-hoc rather than prospectively pre-registered, consistent with our own disclosure standard.

232/232 trials with the process pinned to a fixed CPU core fell under 150  $\mu\text{s}$ , versus 70.1% 150  $\mu\text{s}$  in the original (unpinned) campaign (Mann-Whitney  $p = 1.8 \times 10^{-4}$ , Hodges-Lehmann shift  $-184.4 \mu\text{s}$ ). A same-session unpinned control block showed no difference from the pinned blocks ( $p = 0.673$ ), ruling out session-to-session drift as a confound. IRQ-routing is excluded on architectural grounds (Tegra’s chained GPIO IRQs are not steerable; INT1 is fixed to CPU0 regardless of process placement). Process affinity is not supported by the follow-up data; the anomaly’s non-reproduction across sessions — in both the pinned and same-session unpinned control blocks — means no software-side candidate can be excluded from these data alone.

We report this honestly as a **systematic negative result**, not a resolved anomaly: the root cause of the idle-session multimodality remains unidentified. We do not believe this affects the paper’s primary conclusions, which rest on the unpinned confirmatory campaign data throughout, and we

have added the multimodal-distribution caveat and candidate-mechanism discussion (bus arbitration, gpiod chardev poll state, MLC-cadence interaction) as a labeled exploratory finding.

**Manuscript changes:** §V.B (multimodal distribution, explicitly labeled exploratory). Full candidate-mechanism discussion, including the IRQ-routing hypothesis (excluded on architectural grounds), process affinity (not supported by the data), and the three remaining untested candidates, is provided in the supplementary material (S3) referenced from this letter, since it did not fit within the 4-page limit alongside this revision’s other required additions.

---

### Point 3 — Absence of a wire-level timing diagram

*“For a ‘wire-level’ study, the absence of logic-analyzer traces, or a timing diagram showing INT1/D1 edges and the three I<sup>2</sup>C transactions, is a glaring omission. A single figure would have added enormous credibility.”*

**Response:** We agree, and the omission is corrected with a measured SDA/SCL figure, not a schematic. The figure below is rendered by `code/analysis/fig_wire_timing.py` directly from the archived Saleae export for block b005 (mlc-idle, nvpmode mode 3) in the evidence package (Zenodo DOI 10.5281/zenodo.22950121). The four lanes are INT1 (D0), SDA, SCL, and the decision strobe D1. The decoder requires and finds exactly the four wire frames of the bank-switch read in the shown window: T1 W[01,80] (embedded-functions bank ON), T2 W[70] + repeated-start read of MLC0\_SRC (=0x04, motion), and T3 W[01,00] (bank OFF). For the shown window (t = 22.906 s): INT1→first-byte scheduling 57.4 μs, complete three-transaction triple 383.4 μs, last-STOP→D1 strobe 27.5 μs, D0→D1 468.3 μs, with no foreign frame inside the triple. Campaign statistics remain in Table I and S4; this figure is the wire-level existence proof for the protocol those statistics decompose.

**Manuscript changes:** §VI.A pointer sentence added; the measured capture is provided as Fig. S4 in the supplementary material and is embedded in this response, generated reproducibly by `code/analysis/fig_wire_timing.py` from the archived capture; the extended candidate-mechanism discussion remains in S3 to preserve the 4-page limit.

*Fig. R1. Measured wire-level decision window for block b005 (mlc-idle, nvpmode mode 3): INT1 (D0), SDA, SCL, and decision strobe D1. Decoded frames: T1 W[01,80] bank ON; T2 W[70] + repeated-start read MLC0\_SRC=0x04; T3 W[01,00] bank OFF. Shown-window decomposition: scheduling 57.4 μs, triple 383.4 μs, strobe 27.5 μs, D0→D1 468.3 μs; no foreign frame inside the triple.*

---

### Point 4 — n=532 vs. n=534 discrepancy (mlc/i2c-contention)

*“In Table 1, the MLC/I<sup>2</sup>C-contention cell is reported as n = 532... However, Section V.A (H7) reports 534/540... Either the inclusion count and the stability count are based on different criteria... or Table I contains an error.”*

**Response:** These are two different, correctly-applied criteria, not an error. n = 532 is the **latency-included** count (540 candidate trials minus 8 exclusions: 5 multiple-D1-edge trials, 2 multiple-D0-before-D1 trials, 1 no-D1 trial), used throughout Table I and the confirmatory latency statistics. n = 534 (540 minus 6 exclusions) is the **single-D1-rising-edge stability count**, which

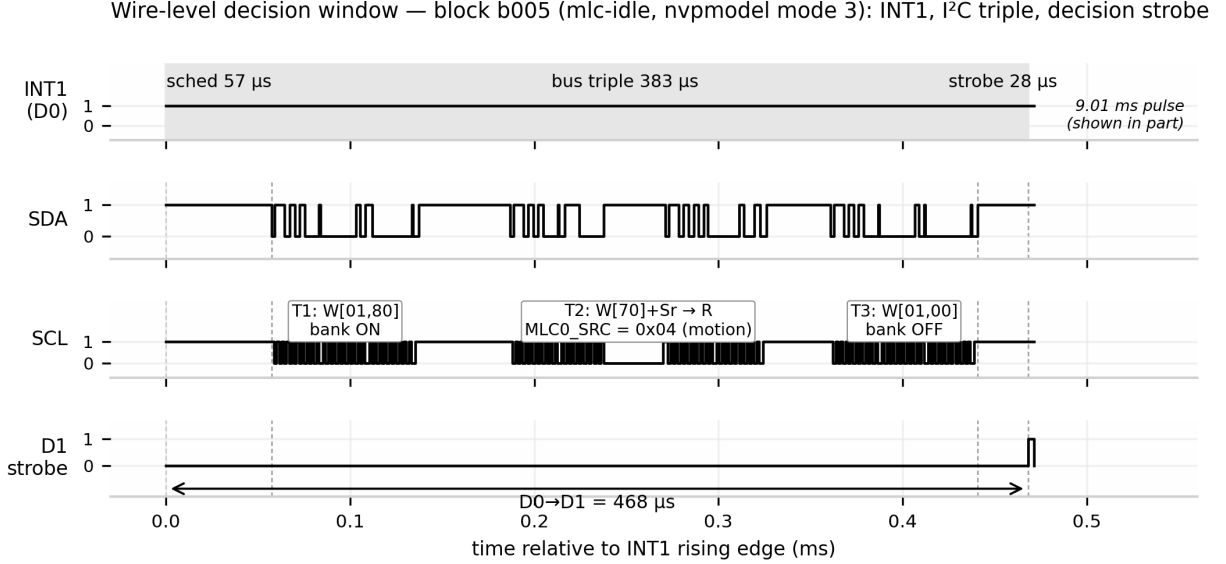


Figure 1: Fig. R1. Measured wire-level decision window for block b005 (mlc-idle, nvpmode mode 3): INT1 (D0), SDA, SCL, and decision strobe D1. Decoded frames: T1 W[01,80] bank ON; T2 W[70] + repeated-start read MLC0\_SRC=0x04; T3 W[01,00] bank OFF. Shown-window decomposition: scheduling 57.4 μs, triple 383.4 μs, strobe 27.5 μs, D0→D1 468.3 μs; no foreign frame inside the triple.

is the specific criterion H7 tests. The two counts differ because they answer different questions from an overlapping but non-identical set of exclusion categories. Both criteria and both resulting counts are now stated explicitly in the manuscript.

**Manuscript changes:** §III.D and §V.D now state both criteria explicitly and distinguish them (§III.D’s inclusion criterion additionally requires an unambiguous single D0 edge, which the H7’ stability criterion does not); the specific per-category exclusion breakdown given above remains stated only in this response letter, not in the manuscript text itself.

## Point 5 — No host-vs-MLC energy comparison

*“The paper measures energy only as a manipulation check for CPU stress... and never compares host-inference energy vs. MLC-inference energy... The conclusion that the host is the better choice at ‘only 359 μs earlier at idle’ cannot be evaluated without the energy axis.”*

**Response:** We evaluated host-vs-MLC energy at the Jetson platform-power level (INA3221, VDD\_CPU\_GPU\_CV and VDD\_IN rails):

Jetson platform power (VDD\_CPU\_GPU\_CV) differs between host and MLC pipelines: +31.1 mW [27.8, 34.4] at idle, +7.3 mW [3.9, 10.7] under I<sup>2</sup>C contention, and −28.8 to +14.6 mW under CPU stress (95% CI includes zero). This comparison was added post-hoc in response to review; per the pre-registration’s standing rule (v7.13), it is labeled as such rather than presented as part of the original confirmatory design. The idle

and i2c-contention confidence intervals exclude zero (Mann-Whitney  $p = 8.4 \times 10^{-2}$  and  $p = 5.5 \times 10^{-2}$  respectively); the stress condition’s does not, despite  $p = 1.4 \times 10^{-2}$  — expected at this sample size, where the Mann-Whitney U test is sensitive to distributional differences beyond central tendency, which is why the confidence interval, not the p-value, is the practically relevant statistic throughout. VDD\_IN replicates the idle effect at +37.3 mW [33.2, 41.4]. On the VDD\_IN rail, the idle effect (+37.3 mW) is approximately 0.7% of the 5,206 mW idle platform draw (VDD\_IN baseline, H6) — on this platform, the host pipeline’s latency advantage costs comparatively little on the energy axis; the open question is whether this holds for MCU-class hosts, which we did not test.

These are whole-platform power measurements, not an isolated host-classifier-vs-MLC comparison — the sensor’s own current draw is not separately instrumented, and we did not repeat measurements on a representative MCU-class host (e.g., STM32); both are natural directions for follow-on work rather than achievable within this revision’s window. We did not pursue direct sensor-level current measurement (e.g., via a Nordic PPK2) within the revision window; the platform-level comparison above directly answers the reviewer’s core concern that no host-vs-MLC energy comparison previously existed.

The full numbers, including all three conditions and both power rails, are provided in the supplementary material uploaded with this revision (S2), since the full comparison did not fit within the 4-page limit as in-text prose alongside this revision’s other required additions.

**Manuscript changes:** None in-text beyond what is already noted (H6 remains the CPU-stress manipulation check, unchanged and now explicitly distinguished from this comparison — see Reviewer 2, Point 5, below).

---

### Additional points from the Key Weaknesses summary

**Dominant-axis scoping.** We agree the original used “dominant” on two axes. The revision scopes it: on the wire-level D0-to-D1 axis, the read protocol dominates (§VI.A); on the full stimulus-to-decision axis, the 706.5 ms cadence is the structural floor and wire-level differences are second-order (§VI.C). Both statements now appear with their axis attached.

**Trigger semantics.** Disclosed in §III.B: the host pipeline’s D0 (INT1/DRDY) fires on every accelerometer sample; the sliding window is maintained continuously via an  $O(1)$ -per-sample circular buffer, amortized across all preceding edges. On the specific edge that completes the 75-sample window, the measured D0-to-D1 interval additionally contains feature computation (two full passes over the window plus a peak-to-peak scan), decision-tree evaluation, a binary-state comparison, and the GPIO write — real per-decision compute, not the ~721 ms window-accumulation time itself, which is not part of the measured interval. This is asymmetric with the MLC pipelines, whose D0 is itself the decision-ready signal (§III.B): their on-chip classification is complete before D0 fires and lies entirely outside the measured D0-to-D1 interval, while the host’s does not.

---

## Response to Reviewer 2

### Point 1 — Is INT1 configured identically across pipelines?

*“Clearly state whether INT1 is configured identically during host, MLC and mlc-binary experiments.”*

**Response:** It is not, by design, and this is now stated explicitly: the host pipeline sets INT1\_CTRL = 0x01 (accelerometer DRDY, 208 Hz), interrupting on every sample; the mlc and mlc-binary pipelines set INT1\_CTRL = 0x00 and instead route the MLC’s decision-ready signal to INT1 via MD1\_CFG = 0x02. This asymmetry is intrinsic to each pipeline’s decision source (raw sample vs. embedded classifier output) and is now disclosed directly rather than left implicit.

**Manuscript changes:** §III.B, new paragraph.

---

### Point 2 — Report p95, p99, and max for every cell

*“Report p95, p99 and maximum latency for every cell, not only medians/IQRs. This is particularly important given the multimodal distributions.”*

**Response:** A full percentile table (n, median, p25–p75, p95, p99, max, and mean for all nine pipeline×condition cells) is provided in the project’s supplementary material (S1), computed with a single canonical method (linear interpolation, the numpy default) applied consistently throughout. During this revision we identified and corrected an internal inconsistency: the originally submitted Table I used a different interpolation convention (nearest-rank) than subsequent analysis used elsewhere, producing small discrepancies in several p25/p75/p95 values (on the order of 0.1–2  $\mu$ s). This has been corrected throughout the manuscript for internal consistency; Table I’s IQR values in the revised manuscript now match the canonical method used in the supplementary percentile table.

**Manuscript changes:** Table I values corrected for method consistency; full p95/p99/max table provided in the supplementary material uploaded with this revision (S1).

---

### Point 3 — Logarithmic or common-axis figure for Fig. 1

*“Consider reporting latency on a logarithmic or common y-axis in an additional figure; the different scales in Fig. 1 make direct visual comparison difficult.”*

**Response:** Fig. 1 has been redrawn with a single shared log-scale y-axis across all three pipeline panels, replacing the original three independently-capped linear axes. Box position is now directly comparable across panels without per-panel capping or “outliers above cap” text annotations — every data point is shown in-frame at true scale.

**Manuscript changes:** Fig. 1 replaced; caption updated to describe the shared-axis, no-capping presentation.

---

#### Point 4 — Are the three I<sup>2</sup>C transactions necessarily required?

*“Clarify whether the three I<sup>2</sup>C operations are necessarily required for every application or whether software architecture, cached bank state, SPI access, interrupt configuration, or different LSM6DSOX-family devices can change this overhead.”*

**Response:** We address each of the five variables named:

- **Software architecture / cached bank state:** The three-transaction bank-switch cost reflects this implementation’s choice to restore the user bank after every MLC0\_SRC read, not an unconditional silicon requirement. An application that tolerates remaining in the embedded-function bank between reads, or that caches bank state and defers the switch-back until a user-bank register is actually needed, could avoid at least one of the three transactions; we did not implement or measure this variant.
- **SPI access:** Attempted on this platform before an unrelated hardware bring-up failure led us to adopt I<sup>2</sup>C (pre-registration amendment, 2026-05-01). SPI would reduce the fixed per-transaction protocol overhead of I<sup>2</sup>C register reads generally, but we did not measure this pipeline over SPI and report no quantified speedup.
- **Different LSM6DSOX-family devices:** Successor parts (LSM6DSV16X, LSM6DSO32X) retain the same MLC and bank-switched register architecture (§II.A), so the overhead is not specific to this part and should recur on those as well.
- **Interrupt configuration:** We find no basis in AN5259, the datasheet, or our own measurements for a claim that interrupt configuration changes this overhead, and make none.

**Manuscript changes:** §VI.D expanded to cover the software-architecture/cached-bank-state, SPI, and successor-parts points explicitly, within the page budget; the interrupt-configuration point (for which our answer is that we found no basis for a claim either way) is addressed here rather than in-text, since it did not fit within the 4-page limit alongside the other required additions.

---

#### Point 5 — H6 vs. a genuine host/MLC energy comparison

*“H6 measures increased Jetson input power under CPU stress; it does not demonstrate an energy comparison between the host classifier and the MLC. Please make this distinction explicit.”*

**Response:** This distinction is now explicit. H6 remains, unchanged, the CPU-stress manipulation check: a single-condition comparison (stress vs. non-stress) confirming that the CPU-stress condition registers on the power axis, independent of pipeline. The newly added post-hoc comparison (see our response to Reviewer 1, Point 5, above) separately measures host-vs-MLC platform power across all three conditions — idle, contention, and stress — and is the comparison that directly answers this reviewer’s request. Both are now clearly distinguished in the manuscript text and in the supplementary material.

**Manuscript changes:** §V.A (H6 description clarified as a manipulation check); energy-comparison distinction provided in the supplementary material uploaded with this revision (S2).

## Point 6 — Graphical abstract wording

*“The graphical abstract should likewise be worded carefully so that the 2.1–2.3× result is understood as the measured D0-to-D1 configuration rather than universal on-sensor-versus-host inference performance.”*

**Response:** The graphical abstract headline has been rescoped to name the specific platform and protocol rather than implying a general claim:

*“On the LSM6DSOX (I<sup>2</sup>C, Jetson Orin Nano), host inference is 2.1–2.3× faster than the on-sensor MLC; the I<sup>2</sup>C bank-switch read protocol, not classification, dominates.”*

**Manuscript changes:** `make_graphical_abstract.py`, title text.

---

## Point 7 — Overhead decomposition

**Response:** We can decompose two of the five requested components directly from wire-level data: `mlc-binary` isolates the kernel/gpiod floor (49.4 μs under contention), and `mlc-minus-mlc-binary` isolates the full I<sup>2</sup>C read path (476 μs uncontended, 1,276 μs contended) — the increase under contention (~800 μs) is attributable to bus-arbitration delay specifically, since Linux scheduling and GPIO-write cost are shared between `mlc` and `mlc-binary` and cancel in the subtraction. Individually separating I<sup>2</sup>C transaction time from bus-arbitration time, or confirming the silicon’s own classification time, would require SDA/SCL bus traces across the full campaign; this study captures SDA/SCL only for the single representative trial shown in S4, which cannot support a statistical decomposition; a campaign-wide trace is a direction for follow-on work.

---

## Point 8 — General-statements audit

**Response:** We audited abstract, introduction, results, and conclusion: the 2.1–2.3× figure appears only scoped to this platform and the D0-to-D1 configuration; the stress-condition 1.6× is reported separately (§VI.A); §VI.D states platform/protocol/ODR specificity explicitly.

---

## Additional corrections made during this revision

In the course of addressing the points above, we identified and corrected the following, none of which were raised directly by either reviewer:

1. **MLC operating rate:** Corrected from an erroneous 26 Hz (ST example text and the MEMS Studio `gyroscope_odr` field, neither of which is `EMB_FUNC_ODR_CFG_C.MLC_ODR`) to the verified 104 Hz configured rate, at all four sites in the manuscript where it appeared (Introduction, §II.A, §III.A, §V.C), with the derived 706.5 ms cadence figure and its “one-quarter of the window period” explanation corrected accordingly (nominal  $75/104\text{ s} = 721.2\text{ ms}$ ).
2. **Headline speedup scoping:** The original submission stated the 2.1–2.3× speedup held “under every tested condition.” We found this did not hold under CPU stress (measured ratio 1.58–1.6×) and corrected the claim to be scoped accurately to idle and I<sup>2</sup>C-contention conditions, with the stress-condition result now stated and explained separately (§VI.A).

3. **Percentile-computation method:** Corrected an internal inconsistency between the interpolation method used in the originally submitted Table I and the method used elsewhere in the analysis, as noted above (Reviewer 2, Point 2).
4. **Hardware description:** The original submission identified the IMU breakout as the STEVAL-MKI197V1; the board used is an Adafruit LSM6DSOX breakout (PID 4438; onboard 3.3 V regulation, 10K I<sup>2</sup>C pull-ups). The sensor IC, I<sup>2</sup>C address, bus speed, and all electrical configuration are unchanged; no results are affected.

We disclose these here in the interest of full transparency about all changes made between submissions, consistent with the pre-registration integrity standard the paper itself argues for.